VisionAI — Guía para la defensa

Activación, modelos, arquitectura, confianza y rendimiento

Preparado a partir del código y la documentación del proyecto. Revisión: 9 de noviembre de 2026.

1. ¿Qué función de activación usamos?

Respuesta para la defensa: «Los modelos YOLOv8 que usamos emplean principalmente SiLU, también llamada Swish, en sus bloques convolucionales. En la salida de clasificación utilizan sigmoide, que produce valores entre 0 y 1 para las clases» .

La activación introduce no linealidad: permite aprender relaciones visuales complejas. SiLU se expresa como f(x) = x · sigmoide(x). Estas activaciones vienen definidas por la arquitectura adoptada; se verificaron en las exportaciones OpenVINO del YOLOv8 general y del detector de útiles. Esta explicación se refiere a esos modelos, no a todas las operaciones de BlazeFace.

2. ¿Elegimos un algoritmo ya creado?

Respuesta para la defensa: «Sí. Usamos YOLO, una arquitectura de detección de objetos existente. Aprovechamos pesos preentrenados y ajustamos un YOLOv8n con imágenes etiquetadas de lápices, cuadernos y lapiceros. Para rostros usamos BlazeFace mediante MediaPipe».

El detector general utiliza pesos preentrenados. El detector de útiles sí recibió entrenamiento adicional mediante aprendizaje por transferencia. No diseñamos una red desde cero. El ajuste vigente se documenta con 907 imágenes de entrenamiento, 97 de validación y 40 de prueba; la receta utiliza 24 épocas y el optimizador AdamW.

Nuestro aporte incluye integrar los detectores, preparar los datos para el ajuste, construir la interfaz y programar las reglas de puntaje, presencia temporal y rondas. Abrir la cámara solo ejecuta inferencia: mostrar objetos no actualiza los pesos.

3. ¿Se pueden cambiar las neuronas y las capas?

Respuesta para la defensa: «Sí, se puede modificar la arquitectura, pero la aplicación no ofrece un control para hacerlo durante su ejecución. Cambiar capas o canales requiere preparar una arquitectura compatible, entrenarla o ajustarla y volver a exportarla».

En redes convolucionales es más preciso hablar de capas, filtros y canales que de una cantidad simple de neuronas. Una red mayor puede representar patrones más complejos, pero consume más memoria y procesamiento y no garantiza mejores aciertos. Una menor puede ser más rápida y perder capacidad para ciertos objetos.

Cambiar la resolución de entrada o el umbral de confianza no cambia la cantidad de capas. Cambiar entre modelos ya preparados tampoco exige entrenarlos de nuevo.

Confianza y criterios de aceptación

4. ¿Por qué usamos ese porcentaje de clasificación?

El término adecuado aquí es umbral de confianza de detección. El valor inicial del proyecto es 35 % (CONF_DEFAULT = 0.35 en config.py).

Respuesta para la defensa: «El umbral decide qué detecciones aceptamos. Con 35 %, dejamos pasar candidatos cuyo puntaje de confianza alcanza ese valor. Buscamos equilibrar la detección de objetos y el descarte de predicciones débiles».

Límite de la justificación: la documentación revisada no demuestra mediante una comparación sistemática que 35 % sea el valor óptimo. Es el valor operativo elegido. Para justificarlo experimentalmente debemos comparar varios umbrales en validación y medir precisión, recall y los errores relevantes para el uso de la aplicación.

Precisión: de las detecciones aceptadas, qué proporción es correcta. Recall o sensibilidad: de los objetos reales, qué proporción logramos detectar. En detección también importa que la caja localice adecuadamente el objeto.

5. ¿Qué pasa si cambiamos ese porcentaje?

Al bajarlo, por ejemplo a 20 %, se aceptan más candidatos. Podemos recuperar objetos reales con confianza baja, pero también admitir más falsos positivos, es decir, detecciones incorrectas.

Al subirlo, por ejemplo a 70 %, se descartan más candidatos. Podemos reducir falsos positivos, pero perder objetos reales: aumentan las omisiones o falsos negativos.

Ejemplo: una laptop con confianza de 60 % pasa el umbral de 35 %, pero se descarta con uno de 70 %. Si deja de aceptarse, también puede dejar de aportar puntos, según el modo y las reglas temporales.

Respuesta para la defensa: «Cambiar el umbral modifica el filtro de aceptación. No cambia los pesos ni vuelve a entrenar la red».

El control de la interfaz permite valores entre 10 % y 90 %. Para rostros existe además un mínimo interno de 50 %: el umbral efectivo es el mayor entre ese mínimo y el valor del control. Bajar el control a 20 % no permite aceptar rostros con confianza menor al 50 %.

6. ¿Qué porcentajes o criterios no debemos confundir?

• Confianza de una detección: puntaje que el modelo asigna a una predicción. Una caja con 90 % no demuestra una exactitud general del sistema de 90 % ni una probabilidad calibrada.

• Umbral de confianza: mínimo para aceptar candidatos; inicialmente 35 %.

• IoU: mide la superposición entre cajas. En la evaluación documentada se usa IoU mínimo de 0.50 para emparejar una predicción con una referencia; es distinto del umbral de confianza.

• Insignia universitaria: se obtiene desde 50 puntos. Es una regla del programa, no un porcentaje de clasificación ni una salida aprendida por la red.

FPS y cambio de modelo

7. ¿Cuántos FPS usa la aplicación y por qué?

Respuesta para la defensa: «La aplicación no tiene una cantidad fija de FPS: procesa tan rápido como permiten la cámara, los modelos y la computadora. Buscamos superar 15 FPS para mantener una detección suficientemente fluida».

FPS significa fotogramas por segundo. El indicador cuenta resultados nuevos aceptados y dibujados, con suavizado para su presentación. Una cámara de 30 FPS no garantiza 30 detecciones por segundo. El temporizador de 1 ms tampoco garantiza 1,000 FPS.

Resultados registrados en un Ryzen 5 3600, con imágenes reproducidas a 640 × 480 y ambos detectores de objetos activos, en modo General:

• fast — YOLOv8n, entrada de 320 × 320: 21.57 FPS.

• detail — YOLOv8n, entrada de 480 × 480: 17.48 FPS.

• accurate — YOLO26s, entrada de 640 × 640: 7.93 FPS.

En modo Universitario, detail obtuvo 18.72 FPS. Cada corrida duró 10 segundos y excluyó los primeros 2 segundos. Son mediciones con imágenes reproducidas; falta repetirlas con la cámara y el equipo de la exposición.

8. ¿Por qué elegimos detail y qué podría cambiar los FPS?

Respuesta para la defensa: «Elegimos detail porque equilibró el detalle de entrada y la velocidad. Con ambos detectores superó 15 FPS en la prueba registrada, mientras que accurate quedó por debajo».

fast y detail usan la misma arquitectura YOLOv8n con distinta resolución de entrada. Menos píxeles requieren menos procesamiento, pero pueden dificultar detectar objetos pequeños. accurate cambia además el modelo a YOLO26s; puede mejorar ciertas detecciones, con mayor costo de procesamiento, sin garantizar mejoras en todas las clases.

Los perfiles cambian el detector general. El detector especializado de útiles conserva su entrada de 640 × 640. Con su carpeta incluida, el perfil inicial es detail. También influyen la cámara, el hardware y la carga del equipo.

9. ¿Qué implica cambiar de modelo?

Respuesta para la defensa: «Cambiar el modelo modifica los pesos y, si elegimos otra arquitectura, también la estructura de la red. Puede cambiar la velocidad, la memoria utilizada, las clases disponibles y los errores. Por eso debemos volver a evaluar los aciertos y el rendimiento».

Cambiar entre los perfiles incluidos no requiere reentrenar. Incorporar otro modelo exige comprobar su formato, compatibilidad de ejecución, clases y correspondencia con el puntaje. Debemos evaluar sus errores y FPS en el equipo real; si no reconoce las clases necesarias, puede requerir entrenamiento adicional y una nueva exportación.

Frase para recordar

«Usamos YOLO con pesos preentrenados y un detector de útiles ajustado. El umbral filtra detecciones; el modelo y la resolución influyen en el costo de cálculo. Cambiar una configuración requiere comprobar sus efectos con datos y en el equipo de uso».

Referencias locales: config.py, detector.py, app.py, scoring.py, DATASET_RECONOCIMIENTO.md, GUIA_DEFENSA.md y archivos XML de los modelos YOLOv8.

Código: datos, cajas y confianza

10. Recorrido de una detección

Cámara → imagen → modelos → clase, confianza y caja → filtros → dibujo → puntaje universitario. La red propone detecciones; nuestro programa decide cuáles mostrar y cómo utilizarlas.

11. ¿Qué son los ID de los datos?

El ID de clase es el número que identifica una categoría dentro de cada modelo. En models/stationery_openvino_model/metadata.yaml: 0 = pencil (Lápiz), 1 = notebook (Cuaderno), 2 = pen (Lapicero). Cada modelo tiene su propia numeración; el mismo ID puede representar categorías diferentes en dos modelos.

En detector.py:63 se consulta model.names y se crea un diccionario nombre → ID: self._key_ids = {name: i for i, name in self.model.names.items() if name in config.LABELS_ES}. Para útiles, se obtiene la correspondencia desde self.stationery.names.

En detector.py:133 se lee r.boxes.cls y se convierte a enteros. Después, key = ids[int(i)] permite recuperar la clave de la clase. No son ID de objetos individuales: el sistema no hace seguimiento de un lapicero específico entre imágenes.

En el dataset, una anotación YOLO contiene: id_clase centro_x centro_y ancho alto. Sus coordenadas están normalizadas respecto a la imagen. tools/merge_stationery_color.py unifica nombres y reasigna los ID. También usa SHA-256 del archivo para detectar imágenes idénticas y evitar conservarlas en distintas particiones; esto no identifica automáticamente fotos diferentes del mismo objeto.

12. ¿Dónde se crea el bounding box?

La red predice la ubicación. En detector.py:127, model.predict(...) produce el resultado y r.boxes.xyxy entrega (x1, y1, x2, y2): esquina superior izquierda y esquina inferior derecha, en píxeles del frame procesado.

Ejemplo: (100, 50, 300, 250) tiene ancho 200 y alto 200. _clip_box(), en detector.py:32, limita las coordenadas a los bordes y descarta cajas sin área. Luego draw_box(), en app.py:115, dibuja con p.drawRect(QRectF(x1, y1, x2 - x1, y2 - y1)).

Respuesta para la defensa: «YOLO predice las coordenadas; nuestro código las valida y QPainter dibuja la caja, el nombre y la confianza». Para rostros, MediaPipe devuelve origen, ancho y alto; se convierten al mismo formato de esquinas.

13. ¿Dónde se obtiene el porcentaje?

La confianza viene de r.boxes.conf en detector.py:132. El texto f"{d.conf:.0%}" en app.py:642 solo convierte, por ejemplo, 0.82 en 82 %. El umbral es otro valor: on_conf_changed(), en app.py:842, calcula self.conf = value / 100 y lo pasa a model.predict(conf=conf). La biblioteca aplica ese filtro.

Respuesta: «El modelo calcula la confianza. Nosotros establecemos el mínimo de aceptación y damos formato de porcentaje al resultado».

Lapiceros y parámetros modificables

14. ¿Cómo incorporamos los lapiceros?

Primero incorporamos imágenes anotadas y unificamos nombres como lapicero y pen. Luego ajustamos YOLOv8n con las clases pencil, notebook y pen. La aplicación carga su exportación OpenVINO como detector especializado, además del detector general.

En config.py:48, LABELS_ES contiene "pen": "Lapicero". En config.py:67, UNI_LIVE_ITEMS contiene "pen": ("Siempre preparado", 5). La primera entrada define el texto; la segunda permite puntuar. Escribir una etiqueta no enseña al modelo a reconocerla: hacen falta pesos entrenados para esa clase.

Detector valida que el modelo de útiles incluya pencil y notebook, y permite además pen. Con el modelo incluido, pen tiene ID 2. El lapicero puntúa en universitario en vivo, pero no en la ronda de 60 segundos: las tablas son distintas.

15. ¿Qué valores pueden cambiar?

• CONF_DEFAULT = 0.35: umbral inicial. El control de interfaz permite 10–90 %. Subirlo descarta más candidatos; bajarlo admite más, incluidos posibles errores.

• FACE_MIN_CONF = 0.50: mínimo interno de rostros. El umbral efectivo es max(0.50, confianza del control).

• MODEL_PATH e IMGSZ: detector general y tamaño de entrada. Pueden cambiar capacidades y FPS. La resolución elegida debe ser compatible con la exportación; no se puede asumir que cualquier tamaño sea válido.

• STATIONERY_MODEL_PATH y STATIONERY_IMGSZ = 640: ruta y entrada del detector especializado. Si falta su carpeta predeterminada y no se proporciona otra, la app inicia sin ese detector; el modelo general incluido no aporta la clase pen.

• PRESENCE_ENTER_S = 0.3 s y PRESENCE_HOLD_S = 0.7 s: confirmación y tolerancia de presencia. Afectan al conteo y puntaje, no a los pesos de la red.

• UNI_THRESHOLD = 50: puntos para insignia. UNI_ROUND_S = 60: duración. UNI_RESULT_S = 5: permanencia del resultado. Modifican reglas del programa, no reconocimiento.

• UNI_ITEMS y UNI_LIVE_ITEMS: clases, puntos y títulos para cada modalidad. Agregar una clase solo funciona si el detector también la soporta. El máximo se calcula a partir de las clases disponibles.

• CAMERA_INDEX_DEFAULT, FRAME_W y FRAME_H: cámara y resolución solicitada. Un índice incorrecto puede impedir recibir imágenes; la cámara puede entregar una resolución distinta de la solicitada.

• LABELS_ES, COLORS y THEME: traducción y presentación. Cambiar un nombre visible o un color no reentrena ni mejora las predicciones. Las claves internas deben seguir coincidiendo con las clases del modelo.

Ejemplo para mostrar al docente

«Un lapicero con confianza de 62 % pasa el umbral de 35 % y dibujamos su caja. En universitario en vivo, tras confirmar presencia, suma cinco puntos. Si elevamos el umbral a 70 %, esa predicción deja de aceptarse; sus puntos desaparecen cuando vence la tolerancia de presencia».

Modo universitario y estabilidad

16. ¿Qué hace General ↔ Universitario?

toggle_mode(), en app.py:828, llama a set_mode("uni" if self.mode == "general" else "general"). set_mode() actualiza self.mode, deshabilita los controles generales de grupos en universitario, muestra el panel correspondiente y reinicia el puntaje actual al entrar. También invalida respuestas pendientes de la configuración anterior.

En tick(), el modo universitario usa groups = {"objetos"} y only_keys = frozenset(self.uni_scorer.items). Esto restringe las clases solicitadas a las que puntúan. El cambio afecta filtros, dibujo y lógica de puntaje. No carga otra red ni vuelve a entrenarla.

17. ¿Qué hace la casilla «Ronda de 60 s»?

En set_uni_round(), app.py:831, live = not enabled. Se actualiza self.live_scoring y se crea UniScorer(self.detector.available_keys, live=live). Se reinician el puntaje actual y su estado temporal, se invalidan trabajos anteriores y se limpia last_dets. Se conserva el mejor puntaje de la sesión.

En vivo: mochila 10, cuaderno 15, libro 20, laptop 25 y lapicero 5. Máximo 75 si todas están disponibles. Cuenta lo presente; al retirar un objeto, sus puntos desaparecen después de la tolerancia. No hay límite de ronda.

Ronda: mochila 10, libro 20, laptop 25 y celular 5. Máximo 60 si todas están disponibles. Empieza con la primera entrada confirmada y conserva cada tipo aunque se retire. Termina al reunir todos o agotarse 60 segundos. El resultado dura cinco segundos y luego vuelve a esperar.

La casilla cambia también la tabla de objetos; no únicamente el reloj. Alcanzar los 50 puntos de insignia no termina por sí solo la ronda.

18. ¿Cómo evitamos el parpadeo de puntos?

PresenceTracker, en scoring.py:7, guarda primera y última observación de cada clase. Confirma presencia si last - first >= 0.3 y la mantiene si now - last <= 0.7. Una sola detección y esperar 0.3 segundos no basta: se necesitan observaciones positivas separadas en el tiempo.

Los huecos breves se toleran. Si las observaciones positivas están separadas por más de 0.7 segundos, la racha reinicia. Un rendimiento muy bajo puede impedir confirmar presencia aunque el objeto siga frente a la cámara.

El puntaje se calcula por tipo, no por cantidad de ejemplares: score = sum(self.items[k][1] for k in collected), en scoring.py:105. Dos lapiceros aportan los mismos cinco puntos que uno. En vivo se usan clases presentes; en ronda, clases acumuladas.

19. Particularidades del resultado universitario

El título depende del objeto activo con mayor puntaje. Una laptop sola puede mostrar «Listo para el aula» con 25 puntos, sin insignia. La insignia requiere score >= UNI_THRESHOLD. El tiempo se comprueba antes de sumar nuevas observaciones: un resultado procesado después del plazo no se acredita a la ronda terminada.

Control interno, modelos y escritorios

20. Variables internas críticas

self.mode decide General o Universitario. self.live_scoring decide puntaje visible o ronda. groups selecciona objetos, personas y rostros; only_keys limita clases. self._collected conserva las clases reunidas en una ronda.

self._generation identifica la configuración de un trabajo. Cuando cambia el modo o la confianza, aumenta la generación. Si llega una respuesta de la anterior, se descarta en on_inference_finished(), app.py:561. Ejemplo: una inferencia enviada con 35 % no se usa después de subir el control a 90 %.

self._inflight impide tener más de una inferencia pendiente. Invalidar un resultado no cancela por fuerza el cálculo: se espera su devolución y luego se descarta. Esto evita acumular imágenes atrasadas y protege los contadores y el puntaje.

21. Otras particularidades del software

La inferencia se ejecuta en un QThread mediante InferenceWorker; la cámara y los widgets pertenecen al hilo de interfaz. Así los controles pueden responder mientras se procesa una imagen.

Para fusionar libro y cuaderno, detector.py:42 calcula su superposición. Si el general devuelve book y el especializado devuelve notebook con IoU >= 0.7, se elimina book y se conserva notebook. La regla no elimina cualquier par de clases superpuestas ni exige que cuaderno tenga mayor confianza.

El aviso de poca luz usa brillo medio de una miniatura en gris y una condición temporal. Avisa; no corrige automáticamente la imagen ni los pesos. Los FPS cuentan resultados nuevos aceptados y dibujados, no simples repintados de una imagen anterior.

22. ¿Usamos cuatro modelos?

La aplicación combina tres detectores: un YOLO general, un YOLO especializado en útiles y BlazeFace para rostros. Se ejecutan según los grupos y clases solicitados; en universitario no se solicita detección de rostros.

El detector general tiene tres perfiles alternativos: fast (YOLOv8n a 320), detail (YOLOv8n a 480) y accurate (YOLO26s a 640). Se elige uno al iniciar. fast y detail comparten arquitectura con distinta resolución; no se ejecutan juntos.

Contando paquetes de objetos disponibles, hay tres exportaciones del general y una de útiles. Eso no significa cuatro detectores de objetos simultáneos. Respuesta recomendada: «Combinamos tres detectores y elegimos un perfil para el general».

23. ¿Reconoce escritorios?

Sí puede detectar algunos escritorios mediante dining table, cuya etiqueta visible es «Mesa / Escritorio» en config.py. No distingue específicamente escritorio de mesa; depende de la predicción del modelo. Aunque LABELS_ES incluye desk, los modelos actuales no aportan esa clase específica.

En General puede mostrarse si está activo Objetos y pasa confianza. En Universitario se excluye porque no está en las tablas de puntaje. Respuesta: «Reconocemos mesas y algunos escritorios mediante dining table; no entrenamos una clase específica de escritorio».